

Sprint 2

Implementação do Simulador em C



Rafael Oliva Soares, Derick Maciel Muniz, Luis Felipe Caleal Macalli

Objetivos

- **Implementar** um simulador de memória cache com políticas de substituição
- **Validar** o desempenho do Perceptron em comparação com o LRU.



Simulador

config.h — Parâmetros da cache (capacidade, bloco, associatividade)

cache.h — Definição das structs: bloco de cache, conjunto via

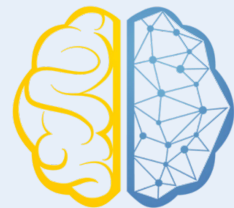
cache.c — Lógica de hit/miss, política de substituição

main.c — Benchmarks e métricas como hit rate



O funcionamento geral do simulador ocorre em etapas:

- Geração de acessos
- Verificação da cache
- Previsão do perceptron
- Cálculo de desempenho



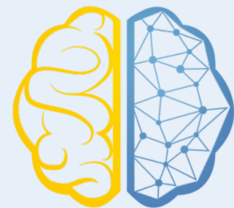
Função Cache_access:

O papel dela é decidir se o dado que o processador quer já está na cache (**Hit**) ou se precisa ser buscado na memória lenta (**Miss**), e gerenciar quem entra e quem sai.

Antes de procurar o dado, a função precisa decodificar o endereço de memória de 32 bits fornecido pelo processador. Ela quebra o endereço em duas partes principais usando operações de bits:

- **index:** Descobre exatamente qual conjunto da cache pode conter esse dado.
- **tag:** Como vários endereços mapeiam para o mesmo conjunto, a tag serve para verificar se o bloco ali dentro é exatamente o que procuramos.

A função percorre todas as vias disponíveis. Ela procura por um bloco que seja **válido** e que tenha a **tag idêntica** à calculada. Se ela encontrar, acontece um **HIT**



Função Cache_access:

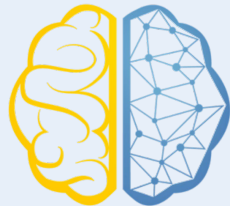
Se o laço terminar e a tag não for encontrada, ocorreu um **MISS**. A cache precisa trazer esse dado de fora, mas antes, precisa descobrir em qual via vai colocá-lo.

A primeira tentativa é amigável. A função percorre as vias procurando por qualquer uma que esteja inválida (!valid). Se achar, aquela posição vazia vira a "vítima" escolhida para receber o novo dado, sem precisar expulsar ninguém.

Se todas as vias estiverem ocupadas (victim == -1), alguém terá que ser expulso para dar lugar ao novo bloco.

No LRU: o bloco que passou mais tempo sem ser tocado. Esse bloco mais velho é escolhido como vítima.

No Perceptron: O bloco que tiver o **menor score**, prevê que ele tem a menor probabilidade de ser usado no futuro



Função Cache_access:

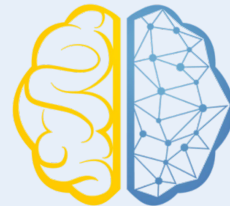
Alocação e Atualização Final:

a função prepara a cache para receber o novo bloco:

Se for LRU: Envelhece todos os blocos válidos do conjunto e zera o contador da vítima.

Se for Perceptron: Atualiza o histórico global do processador (ghr) inserindo um bit 0, sinalizando que ocorreu uma falha de cache (Miss).

via da vítima é atualizada com a nova tag, marcada como válida



Estrutura da cache.h

Vias

[1] Struct Principal: cache	
- num_sets	
- num_ways	
- block_size	
...	
- sets *	
- weights_table **	

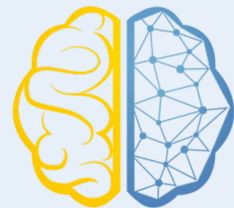
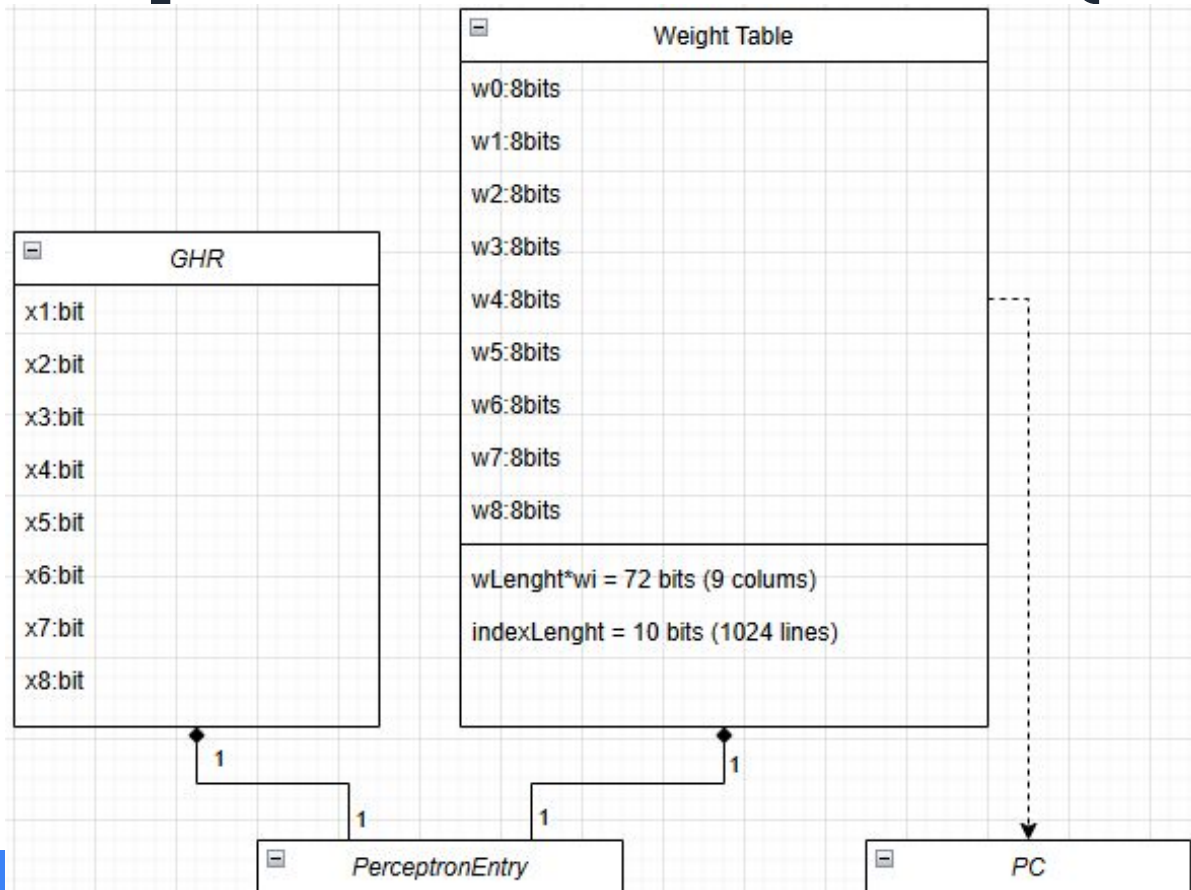
Conjuntos

[2] Array de set (tam: num sets)	
sets[0]	
- num_ways	
- way*	
sets[1]	
- num_ways	
- way*	

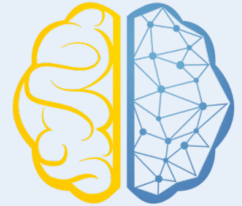
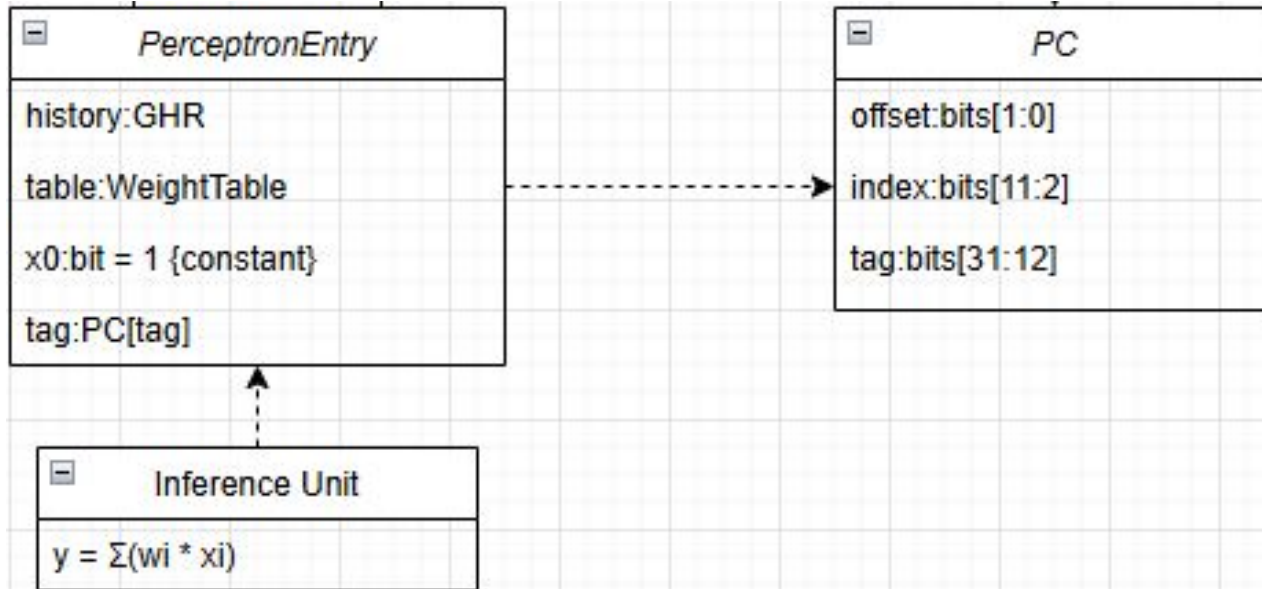
[3] Array de ways (tam: num ways)				
ways[0]	- valid	- tag	- lru_cont	- pc
ways[1]	- valid	- tag	- lru_cont	- pc

[3] Array de ways (tam: num ways)				
ways[0]	- valid	- tag	- lru_cont	- pc
ways[1]	- valid	- tag	- lru_cont	- pc

Mapeamento do AIRA (bits)



Mapeamento do AIRA (bits)



Modelagem do AIRA (dados)

Weight Table (w_i)

índice (Bits do PC)	w_0 (Bias)	w_1	w_2	...
000...001	5	-2	10	
000...010	-15	1	0	
...				

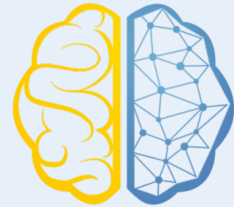
GHR (x_i)

$$u = (w_0 * 1) + (w_1 * x_1) + (w_2 * x_2) \dots$$

Constante

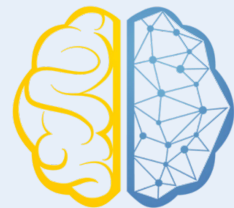
$$x_0 = 1$$

1	1	1	1	0	1	0	0
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8



Modelagem do AIRA (autocorreção)

- Treina quando: $u = \text{miss}$ OU $u \leq \text{threshold}$
- Impede overflow dos pesos



GHR x T

$$T = \text{floor}(1.93 * GHR + 14)$$

GHR: 4, 8, 12, 16, 24, 32

T: 21, 29, 37, 44, 60, 75

$$+W = +T$$

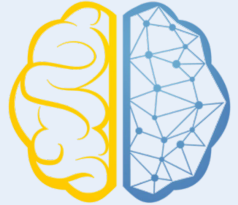


Gráfico de resultados

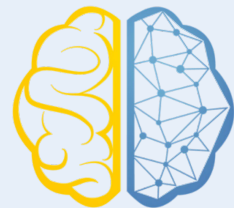
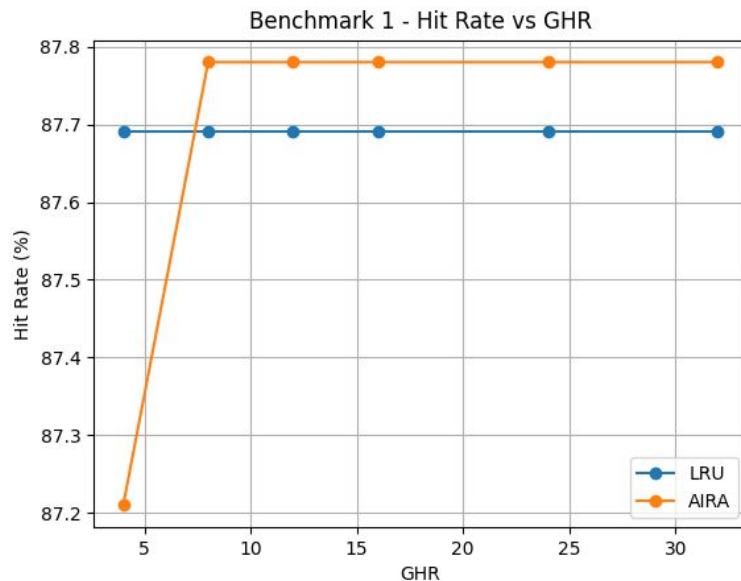


Gráfico de resultados

